

# Informática Musical

Procesamiento de audio digital: NumPy, SoundDevice, SciPy.

Los ejercicios marcados con **(Evaluable)** son prácticas de laboratorio evaluables. El ejercicio 1 es obligatorio; de los restantes evaluables debe seleccionarse **al menos otro** y subir ambos al CV. Los notebooks deben ser **autocontenidos** (incorporar las librerías necesarias).

Todos los archivos entregados tienen que contener el nombre y los apellidos de los miembros del grupo (uno por línea).

1. **(Evaluable)** Implementar una clase `AMmodulator` para **modular en amplitud** (volumen) una señal dada *signal*. La constructora seá de la forma:

```
__init__(self,signal,shape='sin',freq=1,v0=0,v1=1)
```

donde `signal` es la señal a modular (un objeto generador de señal, i.e., que implementa el método `next`) y el resto de parámetros se refieren al modulador:

- `shape`: forma de onda del modulador (puede ser `sin`, `saw`, `square` o `triangle`)
- `freq`: la frecuencia del modulador (en Hz)
- `v0,v1`: los valores mínimo y máximo de la onda moduladora (en el rango  $[0,1]$ , donde 0 es silencio y 1 es el volumen máximo de la señal original). ¿Qué ocurriría si dejamos que `v0,v1` estén fuera de ese rango?

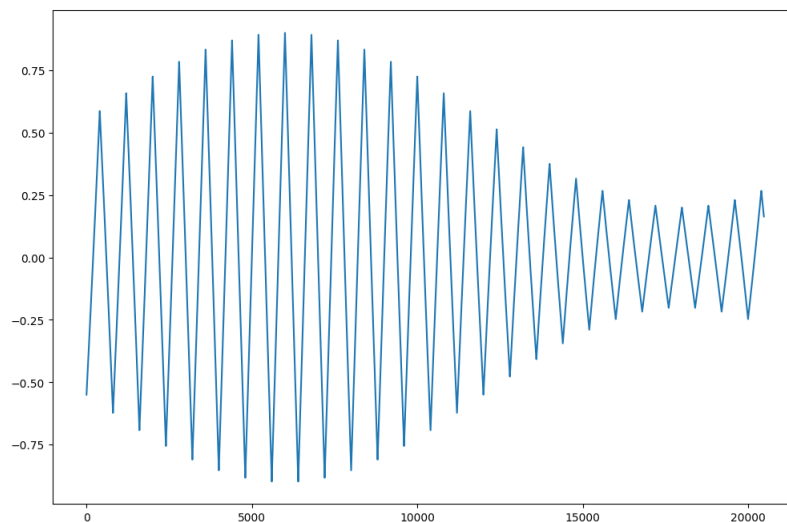
La clase `AMmodulator`, será a su vez un generador de señal, es decir implementará el método `next` para devolver la señal *signal* modulada. De este modo, podremos por ejemplo modular una señal triangular de 440 Hz con un modulador de 2 Hz y amplitud de 0.2 a 0.9:

```
# vamos a modular un oscilador de señal triangular de 440 Hz
signal = Osc(freq=60,shape='triangle')

# con un modulador de 2 Hz, con una amplitud de 0.2 a 0.9
mod = AMmodulator(signal,freq=2,v0=0.2,v1=.9)

# generar 1.5 segundos de señal modulada
time = 1.5
chunks = int(time*SRATE/CHUNK)
msignal = np.empty(0)
for i in range(20):
    msignal = np.append(msignal,mod.next())

plt.rcParams["figure.figsize"] = (12,8) # tamaño de la imagen
plt.plot(msignal)
```



2. Extender la clase `Osc` vista en clase con los métodos *get/set* para poder obtener y modificar dinámicamente el volumen y la frecuencia del oscilador. Después, utilizar la librería `TkInter` para capturar pulsación de teclas e implementar una interfaz para controlar el oscilador anterior en tiempo real: permitirá subir/bajar la frecuencia con las teclas  $[f/F]$  y el volumen con  $[v/V]$ .
3. Implementar una clase `MultiOsc` que permita mezclar señal de los 4 osciladores básicos que hemos visto (`sin`, `saw`, `square` o `triangle`). La constructora recibirá la frecuencia (será la misma para todos los osciladores) y una lista con los coeficientes de mezcla de cada oscilador (en el mismo orden que el listado anterior). Utilizando `TkInter` o los widgets de `IPython`, implementar un interfaz para controlar dinámicamente los coeficientes de mezcla de cada oscilador (con un rango de valores entre 0 y 1). Probarlo con distintas combinaciones de osciladores y coeficientes para obtener timbres interesantes.
4. En este ejercicio vamos a implementar un efecto de modulación de paneo (o balance de canales) para muestras estéreo. En primer lugar, si tenemos un array numpy con una señal estéreo, podemos obtener los canales con `left, right = bloque[:,0], bloque[:,1]`. Una vez realizadas las operaciones deseadas con cada canal, podemos recomponer la muestra estéreo con `bloque = np.column_stack((left,right))`

El paneo es la cantidad de señal que se manda a cada canal. Hay distintas alternativas de implementación (consultar capítulo 9 de *Hack Audio: An Introduction to Computer Programming and Digital Signal Processing in MATLAB*. Eric Tarr. Routledge, 2019.)

Una forma razonable para conseguir *equal power panning* es utilizar la fórmula *square-law panning*. En este caso, dado un valor de paneo  $p \in [0, 1]$ , la amplitud de cada canal viene dada por:

$$a_{left} = \sqrt{1-p}$$

$$a_{right} = \sqrt{p}$$

De este modo, con  $p = 0$  irá toda la señal al canal izquierdo (nada al derecho), con  $p = 1$  irá toda la señal al derecho y con  $p = 0.5$  irá la misma señal a uno y otro. En nuestro caso, queremos modular ese coeficiente (que varíe con el tiempo). Para conseguirlo puede utilizarse la clase `Modulator` del ejercicio anterior o implementar ad-hoc dicho modulador.

5. En este ejercicio implementaremos una clase `SumOsc` que, dada una frecuencia `f` y una lista de coeficientes `coefs=[c0,c1,...,cn]` genera la mezcla de señal de los osciladores `Osc` con frecuencias `[f,2f,3f,...,nf]` con amplitudes `coefs`. Utilizando `TkInter` o los widgets de `IPython`, implementar un interfaz para controlar dinámicamente los coeficientes de mezcla de cada oscilador (con un rango de valores entre 0 y 1). Probarlo con estas combinaciones de coeficientes:
  - `[1,0.5,0.3,0.6,0.2,0.1,0.2,0.1,0.05,0.3,0.2,0.15]`
  - `[1,0,0.4,0,0.2,0,0.1,0,0.05,0.1,0,0.15]`
  - `[0.5,0,0.37,0.04,0,0.1,0,0.012]`

Probarlo con otras combinaciones de osciladores y coeficientes para obtener timbres interesantes.

6. Implementar un programa que permita grabar y reproducir a la vez utilizando la clase `Stream`, con callbacks (en el mismo método callback puede gestionarse la reproducción y la grabación). Para probarlo, tomar un wav de entrada para la reproducción y a la vez grabar otro, que se guardará en un archivo.
7. Implementar un reproductor de *wav* que *normalice* el volumen de la salida, procesando por chunks. Para cada chunk calculará el valor máximo de las muestras en valor absoluto. Con ese coeficiente es fácil calcular el máximo valor por el que puede incrementar el volumen sin saturar la señal (todas las muestras deben quedar en el rango  $[-1, 1]$ ).

Como mejora, el coeficiente multiplicador puede modificarse *suavemente* entre chunks para no generar cambios abruptos de dinámica en el sonido de salida.

8. **(Evaluable)** Implementar una clase `Delay` para hacer una (línea de retardo): recibe una señal de entrada y devuelve esa misma señal, pero retardada en el tiempo una cantidad prefijada de tiempo. La constructora recibirá como argumento el tiempo de retardo en segundos. Tanto la entrada como la salida serán chunks de audio e internamente deberá gestionarse un buffer para producir el retardo.

Para probar el funcionamiento utilizar un audio de entrada (con música o voz) y reproducir simultáneamente dicho audio junto el resultado de retardarlo 0.5 segundos.

Para explotar aún más este efecto, vamos a implementar un *idiotizador*<sup>1</sup>. Para ello utilizaremos un Stream de entrada/salida y reenviaremos la señal de entrada a la salida, con un retardo temporal utilizando la línea de retardo del ejercicio anterior.

9. **(Evaluable)** El *theremin* es un instrumento electrónico basado en osciladores, que consta de dos antenas metálicas que detectan la posición relativa de las manos del intérprete (*thereminista*). Esa posición determina la frecuencia y la amplitud del tono (la frecuencia se controla con una mano y la amplitud (volumen) con la otra). Puede verse un vídeo en <https://www.youtube.com/watch?v=K6KbEnGnymk>

En este ejercicio se implementará un *theremin* con un oscilador básico. Las antenas se simularán con el ratón: la posición horizontal determina la frecuencia (en un rango prefijado, como [50,1500] por) y la vertical, la amplitud en [0,1].

El instrumento debe incluir un pequeño interfaz gráfico con TkInter, que permite detectar fácilmente la posición del ratón en en pantalla:

```
import tkinter as tk

WIDTH, HEIGHT = 800, 600
root = tk.Tk()
root.geometry(f"{WIDTH}x{HEIGHT}")

def motion(event):
    x = root.winfo_pointerx()
    y = root.winfo_pointery()
    print(f'{x}, {y}')

root.bind('<Motion>', motion)root.mainloop()
```

La reproducción se hará mediante callbacks (no bloqueantes) en `SoundDevice`.

Extensiones:

- Utilizar otros modelos de generación, como la síntesis FM. Deberán evitarse o suavizarse los pops al cambiar la frecuencia/volumen.
- Incorporar un *autotune* para obtener solo frecuencias de un conjunto determinado (correspondientes a una escala) y utilizar la posición del ratón para aproximar la nota más cercana de ese conjunto.

---

<sup>1</sup>Véase <https://www.youtube.com/watch?v=zhUDQGWM8kM>